Mississippi Valley State University

CS 341 — Discrete Structures

Lecture 41: Trees With a Top

Rooted Trees and Spanning Trees

Chartrand & Zhang, Section 13.2

Name: _____ Date: _____

The last new material of the course

Two refinements of Monday's idea. Pick one vertex and call it the top: that is a *rooted* tree, and it is the structure behind file systems, parse trees, and binary search. Or take a connected graph and strip it down to a tree: that is a *spanning* tree, and it is what every network-building algorithm computes.

By the end of today you should be able to

1. Use the vocabulary of rooted trees: root, level, height, parent, child.
2. Bound the order of a rooted tree from its degree and height.
3. Find a spanning tree and say how many edges must be removed.

1. Rooted trees

Definitions

A **rooted tree** is a tree with one vertex designated the **root** r.
Level i is the set of vertices at distance i from r. The **height** is the largest level that contains a vertex.
The **parent** of $v \neq r$ is its neighbour one level up; the **children** of v are its neighbours one level down. An **ancestor** of v lies on the r–v path; a **descendant** w has v on the r–w path.

Nothing new is being added

Rooting a tree adds no edges and removes none. It only fixes a reference point, and all the vocabulary is then defined by **distance from the root** — the shortest-path length from Lecture 39.
Every vertex other than the root has exactly one parent, because Theorem 13.11 gives a unique path from r.

Work with this. Root $r = v_1$; edges v_1v_2, v_1v_3, v_2v_4, v_2v_5, v_3v_6.

Level 0: v_1	Level 1: v_2, v_3	Level 2: v_4, v_5, v_6
Height: 2	Children of v_2: v_4, v_5	Parent of v_6: v_3
Leaves: v_4, v_5, v_6	Order: 6	Size: 5

2. How big can a rooted tree be?

If every vertex has at most c children, then level 0 has at most 1 vertex, level 1 at most c, level 2 at most c^2, and so on.

A binary tree ($c = 2$) of height h has order at most

$$1 + 2 + 4 + \cdots + 2^h = 2^{h+1} - 1.$$

That sum is Result 4.7 from Week 11, proved by induction. For $h = 4$ the bound is 31.

Turning the bound around

Reading it backwards: a binary tree holding n items needs height at least about $\log_2 n$. **That is why binary search takes $\log_2 n$ steps** — the height of the tree is the number of comparisons, and the tree cannot be shorter.
A 2^n fact from Week 1, running backwards, becomes an algorithm's running time.

3. Spanning trees

Theorem

A graph G contains a spanning tree **if and only if** G is connected.

How to find one. While a cycle remains, delete one edge of it. Deleting an edge on a cycle never disconnects the graph, and the process stops when no cycle is left — leaving a connected acyclic spanning subgraph.

How many edges come out? A connected graph of order n and size m has a spanning tree of size $n - 1$, so the number removed is

$$m - (n - 1) = m - n + 1.$$

Connected graph	Edges to remove
Order 10, size 15	$15 - 9 = 6$
K_6 (order 6, size 15)	$15 - 5 = 10$
4-regular of order 10 (size 20)	$20 - 9 = 11$

Where spanning trees are used

A spanning tree is the cheapest way to keep a network connected: every vertex reachable, no redundant edge. With weights on the edges, finding the cheapest one is the **minimum spanning tree problem**, solved by Kruskal's and Prim's algorithms in Section 13.3.
Depth-first and breadth-first search each build a spanning tree as they run. That is what the "search tree" in those algorithms is.

Exit ticket

1. A connected graph has order 12 and size 20. How many edges must be removed to leave a spanning tree?

2. What is the maximum order of a binary tree of height 3?

Before Friday

Review for the final. Friday is a cumulative session — come with questions. The review sheet is posted on Canvas.

Self-check (not collected). Section 13.2, Exercises 1, 3, and 5 — odd-numbered, answers in the back.

Homework 14 is due Friday, December 4, 11:59 p.m.

The last assignment of the term.

Questions I still have